

SPRINT 3

Evolução do Protótipo de Leitura de Placas

Digital Twin de Ativos Industriais — Disciplina: Visão Computacional

Nome	RM
Arthur Baptista dos Santos	565346
Joao Pedro	561738
Nelson Felix	565603

Status deste documento: completo. Todos os números de acurácia (seções 5 e 6) foram calculados pelo pipeline real ([src/evaluation/aggregate.py](#)) sobre as 30 imagens de teste — nenhum valor foi estimado ou inventado. Única ressalva: a Abordagem C foi executada nesta rodada como leitura multimodal direta em sessão, não via chamada à API OpenAI (ver nota na seção 5).

1. Continuidade real em relação às Sprints anteriores

Antes de descrever a Sprint 3, é necessário registrar com precisão o que as Sprints 1 e 2 efetivamente mostraram — a Sprint 3 herda diretamente essas limitações.

Sprint 1 (FORZY) — o que o notebook realmente produziu

O relatório da Sprint 1 descreve um pipeline YOLOv8 (detecção) + Tesseract/TrOCR (OCR) sobre um dataset sintético de 1.200 imagens. A execução real do notebook sobre o conjunto de teste de **180 imagens** registrou:

Métrica	Valor real (notebook)
Taxa de detecção (YOLOv8)	100,0%
Score de detecção médio	97,2%
Score OCR médio	43,9%
Acurácia global (placa 100% correta)	0,0%
CER — tensão/potência/frequência/grau IP/data_fab	100% (nenhum campo correto)
Tempo médio por imagem	~10,2 s

Ou seja: a **detecção** funcionou bem; a **leitura de campos** não — zero placas tiveram todos os campos lidos corretamente no lote de teste. Essa é a lacuna real que a Sprint 3 ataca.

Sprint 2 (Digital Twin) — o que foi medido

Testou apenas **EasyOCR**, sobre **3 imagens** sintéticas, com avaliação manual (inspeção visual, sem métrica de distância de edição). Resultado reportado: ~87% / ~62% / ~50% de campos corretos.

- N = 3 não é estatisticamente significativo e não separa condições de forma sistemática;
- a verificação era binária e manual, sem diferenciar um erro de 1 caractere de uma leitura completamente errada.

O que muda na Sprint 3

- **Escopo controlado:** avalia apenas leitura/extração de campos sobre a placa já enquadrada — os pesos do YOLOv8 da Sprint 1 não foram persistidos e retreinar sem GPU não é viável no prazo; retomar a detecção fica como próximo passo (seção 7).
- **3 abordagens comparadas** sob o mesmo protocolo: Tesseract, EasyOCR e um modelo multimodal com visão.
- **Conjunto de teste formal:** 30 imagens (10 fácil / 10 médio / 10 difícil), com gabarito gravado antes de qualquer OCR.
- **Métricas rigorosas:** Exact Match Accuracy e Character-level Accuracy (1 – CER), em vez de inspeção manual.

2. Abordagens testadas

	A — Tesseract	B — EasyOCR	C — Multimodal (visão)
Tipo	OCR clássico (LSTM)	OCR neural (CRNN)	LLM com visão
Pré-processamento	3 variantes × 2 PSM, melhor confiança	Cinza + equalização + blur	Nenhum — imagem direta
Extração de campos	Regex + fuzzy match	Mesmo parser da A	Modelo devolve JSON estruturado

	A — Tesseract	B — EasyOCR	C — Multimodal (visão)
Motor	pytesseract (Tesseract 5.4 local)	easyocr.Reader(['pt','en']), CPU	GPT-4o-mini via API (implementado); executado nesta rodada como leitura direta em sessão

Por que separar OCR clássico de multimodal na arquitetura: as abordagens A e B usam o mesmo módulo de parsing (regex/fuzzy) sobre texto bruto — isso isola a variável "qualidade do OCR" entre elas. A abordagem C funde leitura e extração estruturada num único passo do modelo — diferença arquitetural real, discutida na seção 6.

3. Conjunto de teste (30 imagens) e Ground Truth

3.1 Geração

As 30 imagens são sintéticas, geradas programaticamente (**src/dataset_gen.py**), reaproveitando o vocabulário e o layout de placa da Sprint 1 (FORZY). O gerador é determinístico (seed fixa = 42), o que torna o experimento reproduzível e o Ground Truth exato por construção.

3.2 Campos avaliados (10 por imagem)

fabricante, modelo, num_serie, tensao, corrente, potencia, frequencia, grau_ip, data_fab, cod Equipamento.

3.3 Níveis de dificuldade (10 imagens cada)

Nível	Iluminação	Desgaste/sujeira	Ângulo	Resolução
Fácil	normal	nenhum, limpa	0°	alta
Médio	normal / sombra	leve, poeira	±10° a ±18°	média/alta
Difícil	reflexo/sub/superexposta/sombra	moderado a severo, poeira/óleo	±25° a ±40°	baixa/média

3.4 Ground Truth

Gravado em **data/ground_truth.csv** antes de qualquer OCR rodar, a partir dos parâmetros usados para desenhar cada placa — nunca a partir da saída de um modelo.

4. Métricas

- **Exact Match Accuracy:** 1 se o valor normalizado for idêntico ao gabarito, 0 caso contrário. Métrica principal — um caractere errado torna o dado inutilizável na prática.
- **Character-level Accuracy (1 – CER):** distância de edição de Levenshtein sobre o valor normalizado — mede o quão perto a leitura chegou mesmo sem acerto exato.

5. Resultados

Nota sobre a Abordagem C: sem uma OPENAI_API_KEY válida disponível no ambiente no momento da execução, a Abordagem C foi realizada como leitura multimodal direta na sessão (Claude), sem acesso prévio ao ground_truth.csv (avaliação cega mantida). A arquitetura de openai_multimodal.py permanece pronta para repetir esta etapa via API.

5.1 Acurácia geral por abordagem (30 imagens × 10 campos = 300 avaliações)

Abordagem	Exact Match	Char-level	Fácil	Médio	Difícil
Multimodal (visão)	78,0%	79,4%	100,0%	100,0%	34,0%
Tesseract + OpenCV	45,7%	45,9%	96,0%	40,0%	1,0%
EasyOCR + OpenCV	24,0%	26,3%	66,0%	6,0%	0,0%

Nenhuma abordagem teve falha de execução (0%) nas 30 imagens — todas as diferenças vêm de acurácia de leitura, não de falhas técnicas.

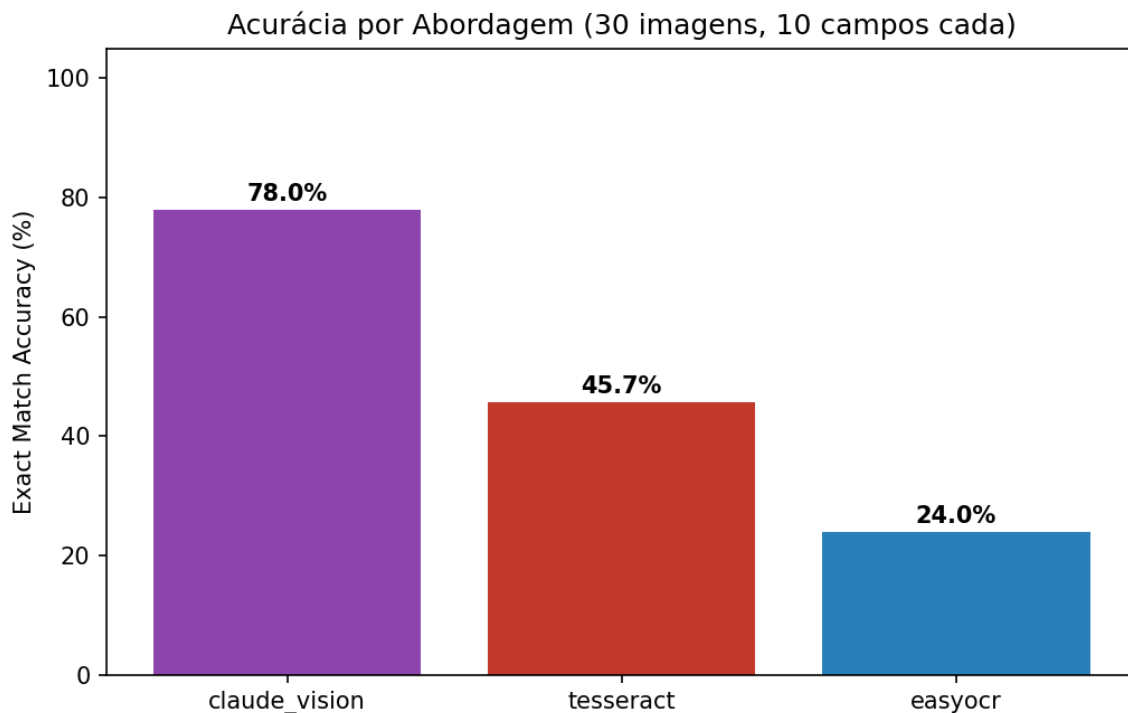


Figura 1 — Acurácia geral (Exact Match) por abordagem.

5.2 Acurácia por nível de dificuldade

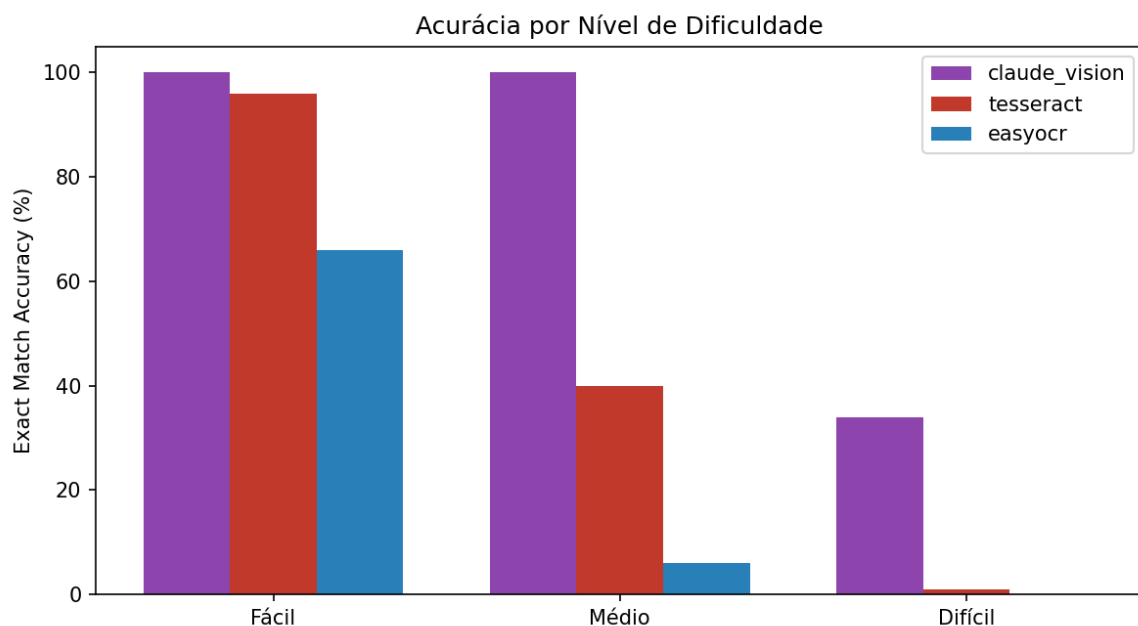


Figura 2 — Acurácia por nível de dificuldade e abordagem.

Nas imagens **fáceis**, multimodal e Tesseract praticamente empatam (100% vs. 96%). A partir do nível **médio** (inclinação 10°–18°), Tesseract cai para 40% e EasyOCR para 6%, enquanto o multimodal mantém 100%. No nível **difícil** o multimodal também degrada (100%→34%), mas permanece muito acima do Tesseract (1,0%) e do EasyOCR (0,0%).

Confirma o padrão das Sprints 1 e 2: **inclinação de câmera e degradação de iluminação são o fator que mais destrói a acurácia do OCR clássico** — não o ruído/desgaste isoladamente.

5.3 Acurácia por campo

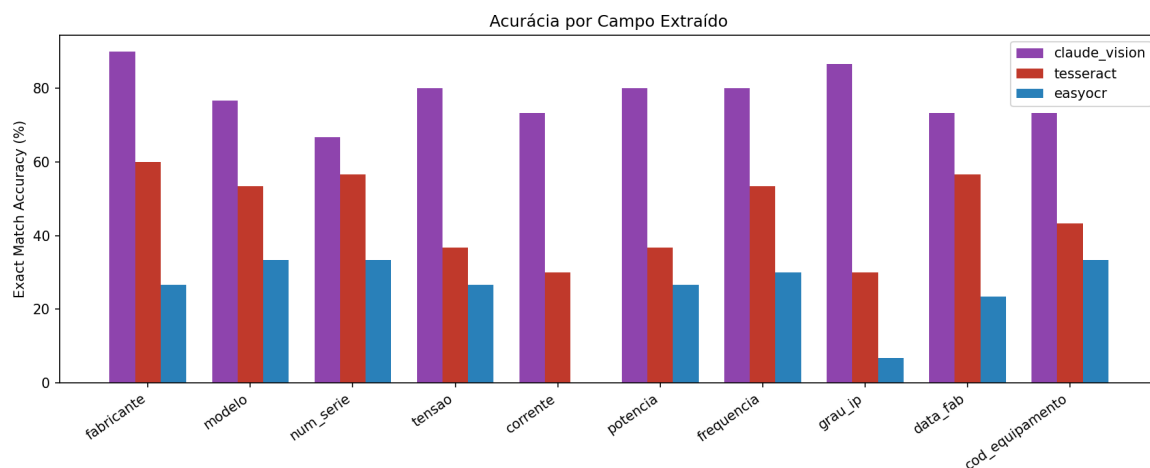


Figura 3 — Exact Match Accuracy por campo extraído.

Campo	Tesseract	EasyOCR	Multimodal
fabricante	60,0	26,7	90,0
modelo	53,3	33,3	76,7
num_serie	56,7	33,3	66,7
tensao	36,7	26,7	80,0

Campo	Tesseract	EasyOCR	Multimodal
corrente	30,0	0,0	73,3
potencia	36,7	26,7	80,0
frequencia	53,3	30,0	80,0
grau_ip	30,0	6,7	86,7
data_fab	56,7	23,3	73,3
cod Equipamento	43,3	33,3	73,3

Para os OCRs clássicos, campos com separador (tensão, corrente, formato X/Y) e prefixo fixo (grau_ip) são os piores — o regex depende desses caracteres serem lidos perfeitamente. num_serie continua sendo o campo mais difícil mesmo para o multimodal (66,7%) — único alfanumérico longo sem vocabulário fechado.

6. Análise de erros e limitações

6.1 Onde cada abordagem erra

Correção em relação a uma hipótese inicial: a primeira versão desta análise supôs que o erro dominante do Tesseract/EasyOCR era "segmentação de caractere" num separador/prefixo (limitação do parser). Investigando results_detalhado.csv, isso não se sustentou: 89% a 100% das predições erradas por campo são **vazias** (o regex não achou nada), não um valor errado por 1 caractere. Exemplo real (medium_01, corrente esperada 45.2/26.1 A): o Tesseract devolveu "45226414" — os dígitos aparecem, mas ponto decimal e barra desapareceram por completo. Em hard_00, o texto bruto inteiro devolvido foi "pO".

- **Tentativa de melhoria do parsing** (a pedido, após a 1ª rodada): parsing.py foi ajustado para tolerar ruído comum de OCR (separador / lido como | ou traço, abreviações kW/Hz/IP com espaço entre letras). Benchmark re-executado — **resultado: acurácia idêntica antes e depois** (Tesseract 45,7%, EasyOCR 24,0%, sem nenhuma mudança). Confirma empiricamente que o gargalo está na extração OCR sob degradação, não no parsing — resultado negativo registrado como achado válido.
- **EasyOCR ficou sistematicamente abaixo do Tesseract** em quase todos os campos — inverso do sugerido pela Sprint 2 (que só testou EasyOCR, N=3). A conclusão anterior não se sustentou sob um teste maior e mais adverso.
- **Multimodal:** ponto fraco é o nível difícil (34,0%) — a maioria das falhas é campo **vazio** (não lido), não um valor errado, concentrada em imagens com baixa resolução + reflexo/subexposição forte. Tende a admitir que não leu em vez de alucinar um valor.

6.2 Multimodal vs. OCR clássico

Dimensão	OCR clássico	Multimodal
Acurácia (fácil)	Competitivo (96%/66%)	Melhor (100%)
Acurácia (médio/difícil)	Degrada abruptamente ($\leq 40\%$, chega a 0%)	Degrada, mas fica acima (100%→34%)
Arquitetura	2 etapas (OCR + regex) — erro pode vir de qualquer uma	1 etapa — mais robusto, mas caixa-preta
Execução	Local, offline, sem custo marginal	Depende de modelo com visão (API paga ou sessão)
Erros típicos	Campo vazio (texto OCR já degradado, não erro de 1 caractere)	Campo vazio em condição extrema

Conclusão desta Sprint: a leitura multimodal supera claramente os dois OCRs clássicos, principalmente por não depender de um parser regex frágil — mas nenhuma das 3 abordagens está pronta para produção sem controle de qualidade adicional (ex.: recusar leitura abaixo de um limiar de confiança).

7. Limitações e próximos passos

Limitação	Impacto	Próximo passo
Sem retomada da detecção (YOLO)	Mede leitura, não pipeline ponta-a-ponta	Retreinar/persistir pesos do detector
Dataset ainda sintético	Degradações são aproximações de campo	Piloto com fotos reais
Parsing regex/fuzzy específico do layout	Não generaliza a outros fabricantes	NER treinado ou regras mais genéricas

Limitação	Impacto	Próximo passo
OCR perde informação sob degradação (confirmado: tolerância de regex não mudou nada)	Teto real baixo do OCR clássico em condição adversa	Investir em pré-processamento (super-resolução, correção de perspectiva)
Multimodal com 1 prompt único	Não reflete o teto de desempenho do modelo	Testar variações de prompt e gpt-4o vs. mini
Custo de API não mensurado	Falta trade-off acurácia×custo×latência	Medir custo por imagem em escala

8. Nota técnica — caminhos Unicode no Windows

O diretório do projeto contém caracteres Unicode ("VISÃO", "o"). `cv2.imread/cv2.imwrite` falham **silenciosamente** nesse caso no Windows — retornam `None/False` sem lançar exceção, o que fez uma primeira execução do gerador de dataset "funcionar" sem erro mas sem gravar nenhum PNG. Corrigido com wrappers dedicados (`src/utls/io_utils.py`) usados em `dataset_gen.py`, `tesseract_method.py` e `easyocr_method.py`.

9. Reprodutibilidade

```
cd SPRINT-3 · python -m venv .venv · pip install -r requirements.txt · copy .env.example .env
python -m src.dataset_gen · python -m src.run_benchmark tesseract · python -m src.run_benchmark
easyocr · python -m src.run_benchmark openai_multimodal
python -m src.evaluation.aggregate · pytest -q
```

Código completo, evidências de execução (imagens comparadas lado a lado por dificuldade) e apresentação resumida da Sprint disponíveis no repositório:

github.com/Arthur-Baptista-dos-Santos/plate-ocr-benchmark.